Deep Learning Overview

Introduction

Deep Learning uses multilayered Neural Networks (NN) for various tasks including regression & classification. Neural Networks are so named from inspiration by the human brain. Neurons (nodes) within the model pass information to each other similarly to brain neurons transmitting electrical impulses.

Comparison with other models:

GLM: Solves linear algebra equation; outputs simple coefficients

GBM: Nonlinear boosted tree method; outputs record level prediction

NN: Nonlinear function approximators trained by gradient descent; outputs record level prediction

Use cases for Deep Learning include claim image damage estimates/classification (Convolutional), claim notes/voice transcript sentiment analysis (Transformers), claim frequency time series analysis (Long Short-Term Memory). There are many flavors of deep learning models built for different use cases that have come up in recent years.

We can also use Neural Networks for regression but its marginal performance improvement over other Machine Learning (ML) models is diminished when working with tabular insurance data. The models being covered in this overview:

1) Simple perceptron model
2) Simple fully connected NN
3) Convolutional NN (CNN)
4) Reinforcement Learning (RL)

As far as performance goes, NN models tend to require a lot of computing power and time to tune hyperparameters & architecture along with the generally inefficient computation algorithm. NN models shine when no other model architecture can perform the task. As always, use the simplest model that can give sufficient predictive power for your purpose.

Note: For self-study I recommend “Learning Deep Learning” by Magnus Ekman. Some images in this overview are taken from the book.

Simple Perceptron Model

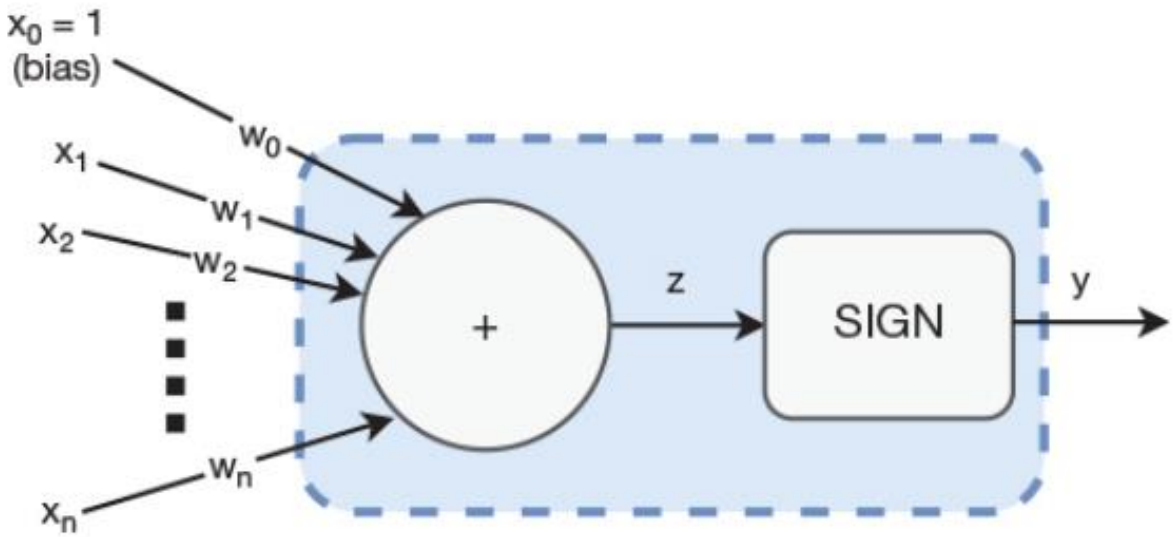


Figure 1-2 The perceptron

$x_0, x_1, x_2, \ldots x_n$ represent the inputs

$w_0, w_1, w_2, \ldots w_n$ represent the weights

$z = \sum_n x_n * w_n$

y = f(z); f is the "activation function"

For the simple Rosenblatt Perceptron model, the output is always -1 or 1. The activation function is the signum (sign) function, which assigns -1 if z <0 and 1 if z >= 0. This simple setup was an early building block of NN architecture.

To turn this setup into a model, let's say we have some labeled training data with 2 independent features + our bias and a target. We initialize 3 random weights and run the perceptron output. Based on the error for each observation, we adjust each weight *incrementally* (small adjustment controlled by the learning rate) until we correctly classify the data.

The below image shows the movement of the model's results toward a correct separation of the positives from the negatives. By the time the algorithm terminates, the blue line has correctly classified each point of the training data.

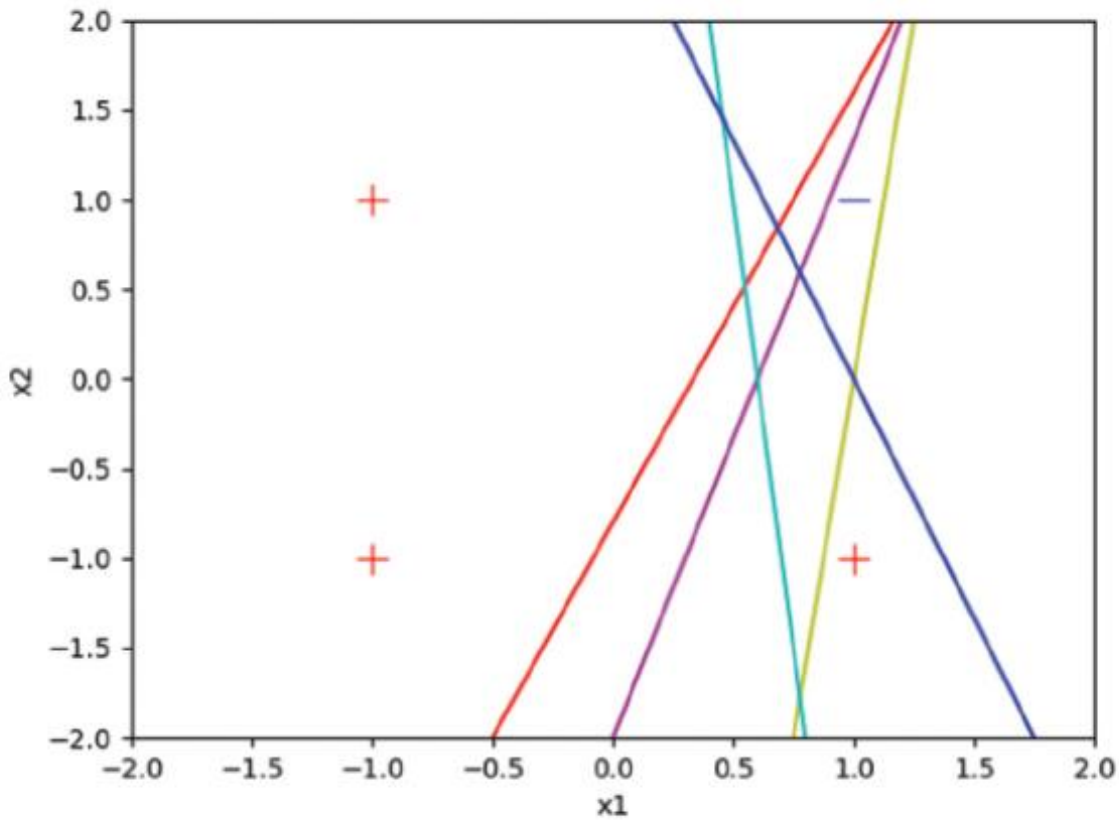


Figure 1-6 Learning process progressing in the following order: red, magenta, yellow, cyan, blue

What if one line of separation is not sufficient to classify the data? If the bottom left point was negative, the algorithm would run indefinitely without convergence. We need a setup that can give us the lines of separation shown below:

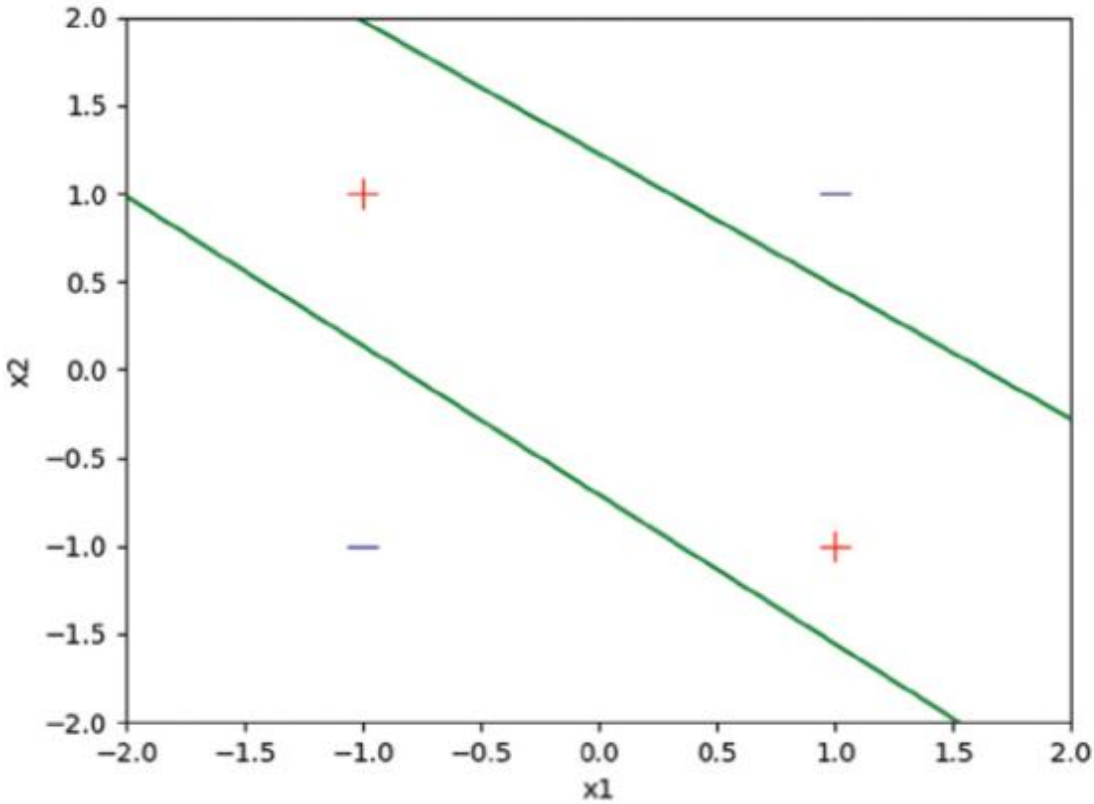
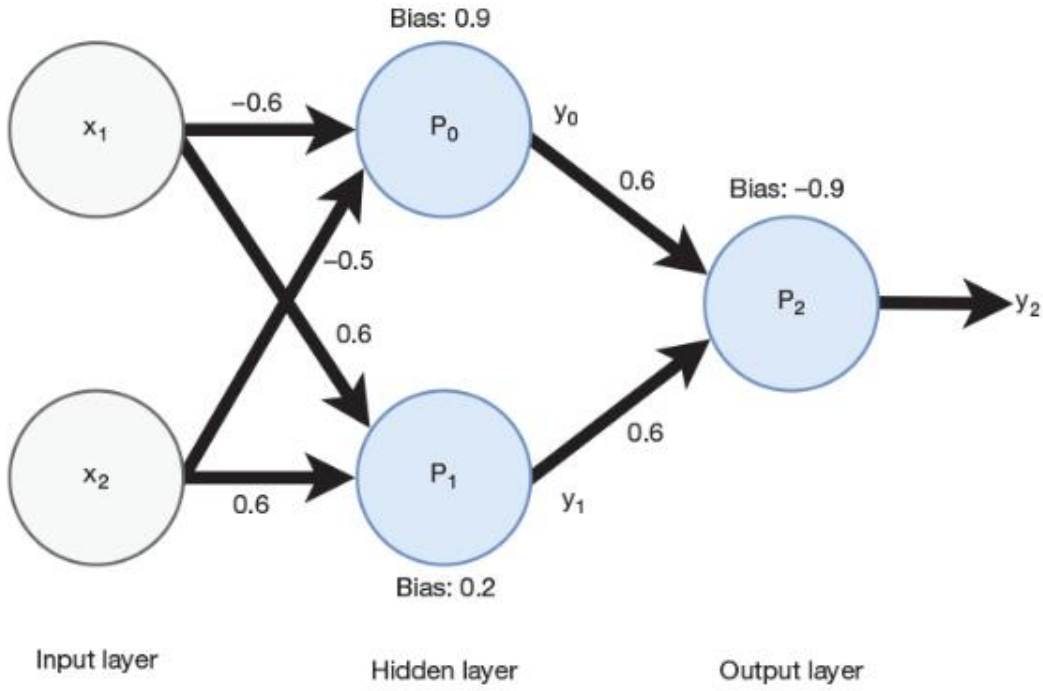


Figure 1-8 XOR output values isolated by two lines

We can add another Perceptron and get the desired behavior.



In this setup, we have a 2 level fully connected feedforward network. Is has 2 levels because we have not just the output layer but also a hidden layer. It is fully connected because each neuron output to connected to all neurons in the next layer. It is feedforward because information only flows forward.

Each x and w are sent to Perceptrons in the hidden layer, which will output either -1 or 1. Those are then sent along with the weights from the hidden layer to the final Perceptron, which outputs the prediction.

In more complex models there can be multiple outputs, multiple hidden layers (Deep Learning), connections both backward and forward, and other convenient transformations to maximize performance for the given task. Any complex data structure can in some way be modeled with a NN.

Fully Connected Neural Network

Before introducing more complex architectures, several core components of neural networks should be briefly discussed.

Gradient Descent – the process of iteratively reducing model error by adjusting model parameters in the direction that most decreases prediction error. In a GLM, this can be done using closed-form optimization. In NN, this is done iteratively through repeated small updates to the model weights. Below are visuals depicting gradient descent in one and 2 dimensions.

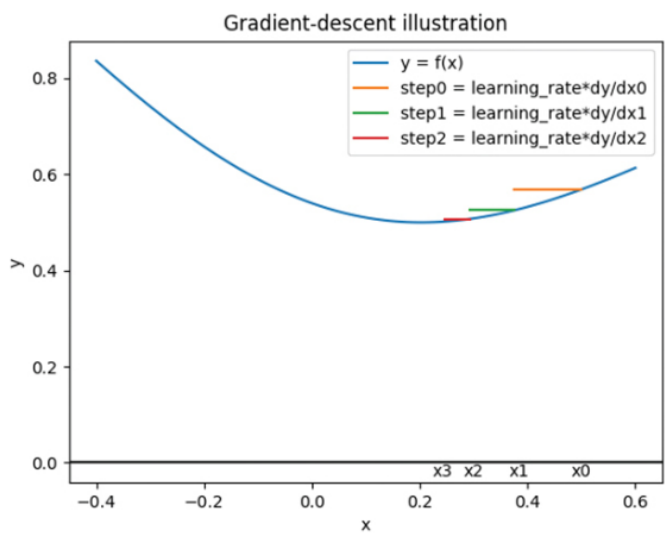


Figure 2-4 Gradient descent in one dimension

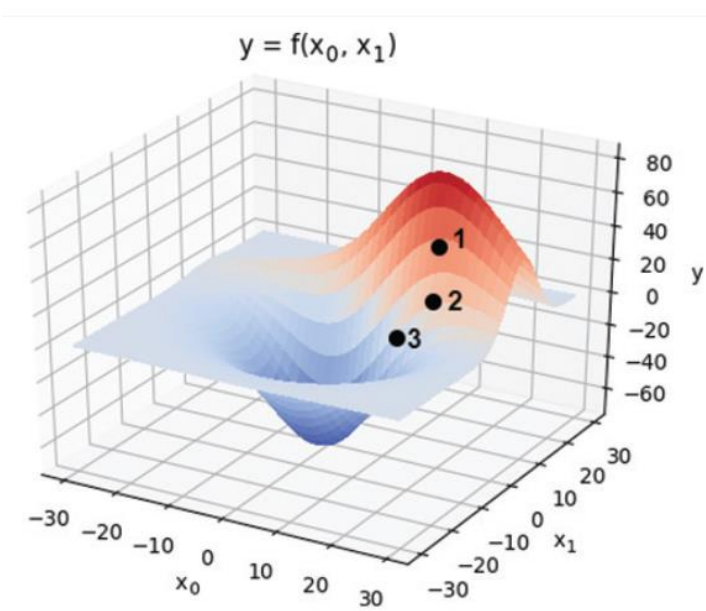


Figure 2-5 Gradient descent for a function of two variables

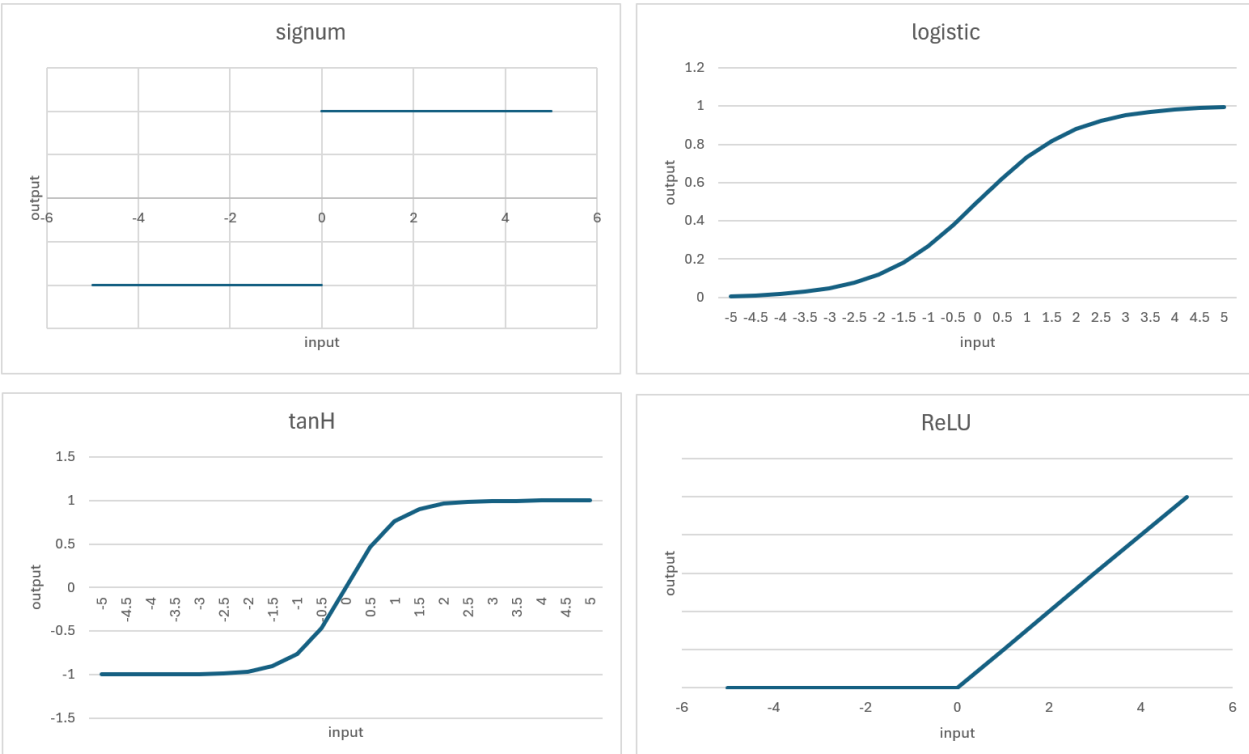
Backpropagation is the algorithm used to train NN and consists of 2 main steps:

1) Forward pass where training data is passed through the NN to obtain predictions.
2) Backward pass where prediction error is propagated backward through the network, allowing each weight to be adjusted using gradient descent.

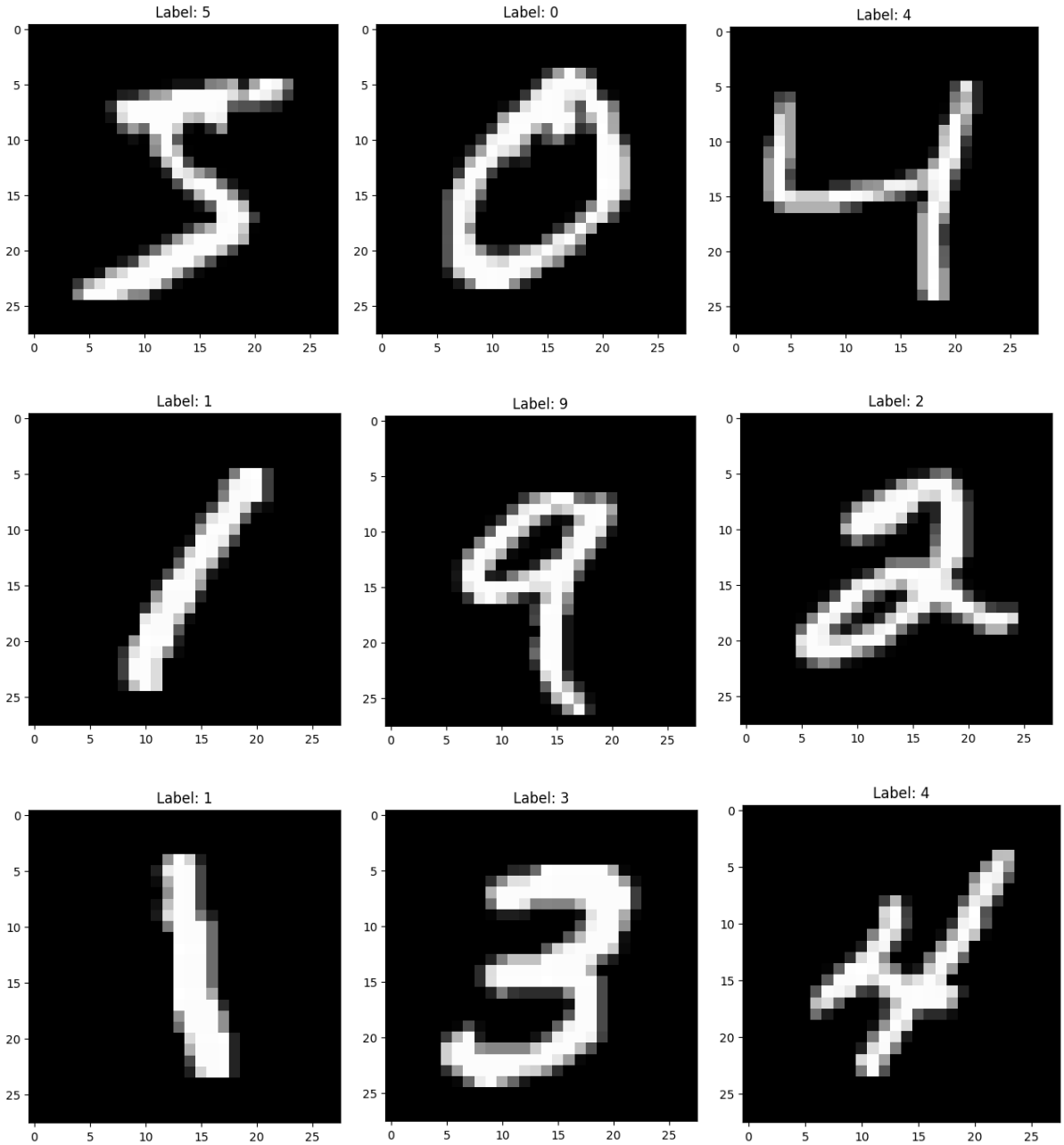
The key here is the iterative process of adjusting weights iteratively until a minimum is reached. For various reasons, this might be a local rather than a global minimum error. Various adjustments to NN architecture can be made to help with this problem and are beyond the scope of this summary.

Activation Function – in our perceptron model we used the signum (sign) function. There are others including Logistic, TanH, ReLU, and more. These additional functions allow modeling of more complex relationships beyond simple linear decision boundaries. What you need to know at a high level is that different types of data and architectures and targets may perform better with different choices of activation function.

Below are visuals of some common activation functions:



Now, let's get into a fully connected NN using the use case of decoding handwritten digits. The Modified National Institute of Standards and Technology (MINST) dataset contains 60,000 training images and 10,000 test images along with labels of what number the digits represent, where each image is 28 by 28 pixels. Some data samples:



Each image is represented by a 28*28 2 dimensional array, which works out to be 784 inputs. Below is a condensed map of the architecture:

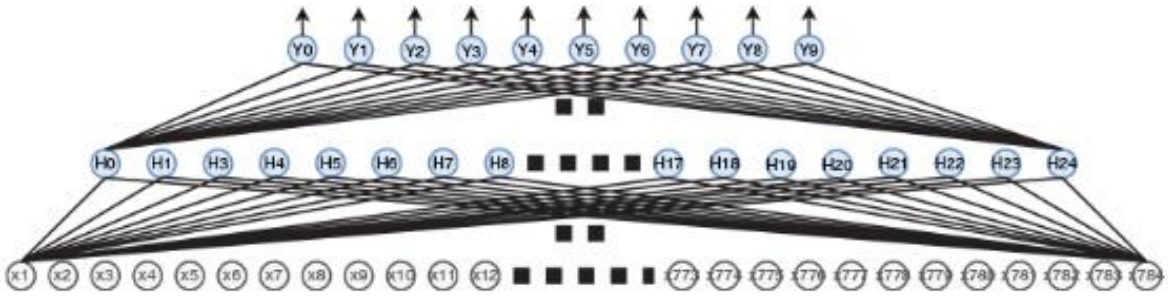


Figure 4-3 Network for digit classification. A large number of neurons and connections have been omitted from the figure to make it less cluttered. In reality, each neuron in a layer is connected to all the neurons in the next layer.

Had we set up just the input layer and the output layer, this would effectively be a multinomial linear regression. This model setup has the following architecture:

784 input neurons

Hidden linear layer with 784 inputs & 128 outputs; ReLU activation function.

128 output neurons produced by 128 weight vectors (784 weights for each using dot product) – conceptualize this as 128 different Perceptron-like neurons with 784 inputs. Note – each does have a bias term.

Hidden linear layer with 128 inputs & 64 outputs; ReLU activation function.

Linear output layer with 64 inputs & 10 outputs (digits 0-9)

With each layer feeding forward into the next, transforming the representation of the data as it goes. In this example we make one pass through the training data, meaning the weights are updated once for every training batch. A complete pass through the entire training dataset is called an *Epoch*.

Even in one Epoch, each training batch (64 in this case) updates the weights which apply to future batches within the Epoch. Each training batch is a chance for the model to learn, even if only one Epoch is run.

Accuracy Statistics allow us to understand how well the model is performing. Our model has accuracy = 94.9% on train and 94.5% on test. It is typical for test accuracy to be lower than train accuracy, and in some cases they are quite a bit different. Our accuracy by digit varies quite a bit due to the difference in homogeneity among people's writing of the different digits:

Digit	Accuracy
0	98.5%
1	98.2%
2	91.6%
3	95.5%
4	95.0%
5	93.4%
6	95.2%
7	95.4%
8	89.4%
9	92.7%

The Confusion Matrix for this data shows us exactly how these digits are being misclassified:

Actual Digit \ Predicted Digit	0	1	2	3	4	5	6	7	8	9
0	965	0	1	1	1	5	3	3	1	0
1	0	1115	1	5	0	1	3	2	8	0
2	11	6	945	21	12	0	3	15	17	2
3	0	0	8	964	1	15	0	11	8	3
4	1	2	8	0	933	0	3	5	3	27
5	7	1	0	19	7	833	11	3	5	6
6	10	3	1	2	17	10	912	0	3	0
7	1	7	15	5	4	1	0	981	0	14
8	4	7	5	27	9	21	10	13	871	7
9	9	5	1	11	23	7	1	14	3	935

Convolutional Neural Network

From here on the focus will be conceptual and very high level. In our fully connected NN we flattened the images into 784 numbers which implies every pixel is independent from every other pixel. This is clearly incorrect and not representative of the reality of image data. Images have structures; edges, shapes, strokes, etc…

We need a model which can represent those structures. Convolutional Neural Network (CNN) models solve this by learning small pattern detectors which scan across the image. They learn small weight grids like this vertical edge detector:

-1	0	1
-1	0	1
-1	0	1

Other filters might learn horizontal edges, curves, corners, circles, and more. Each filter produces a *Feature Map*. Instead of looking at the whole image, neurons are looking for local patterns.

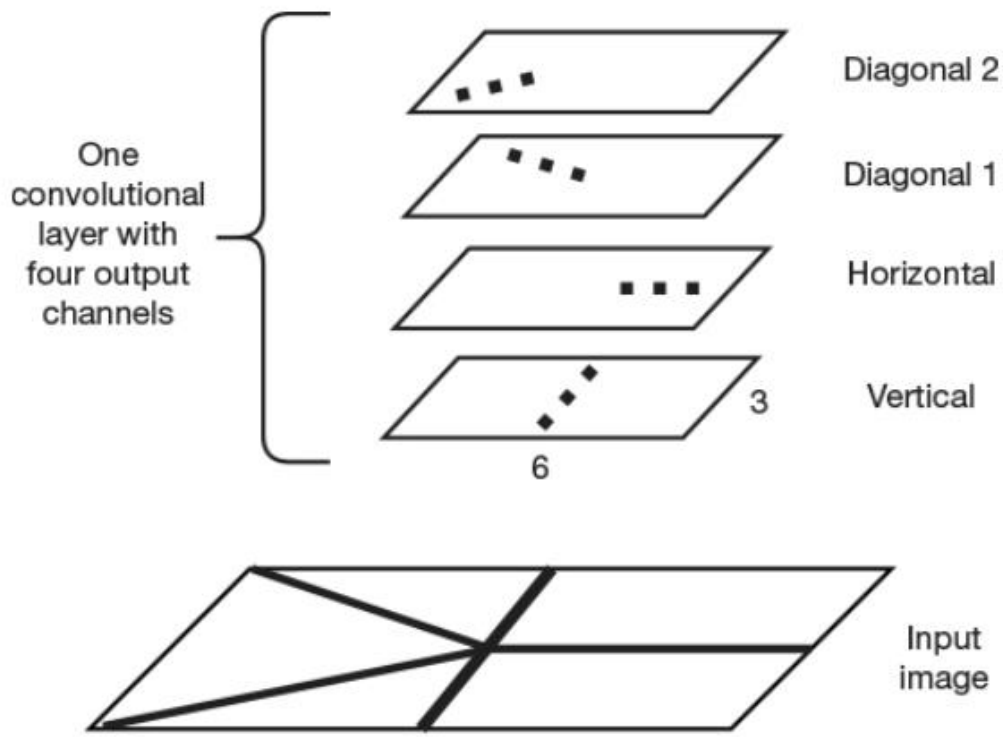


Figure 7-6 A single convolutional layer with four channels and 18 neurons for each channel. Each dot represents an excited neuron.

The layers may work like this:

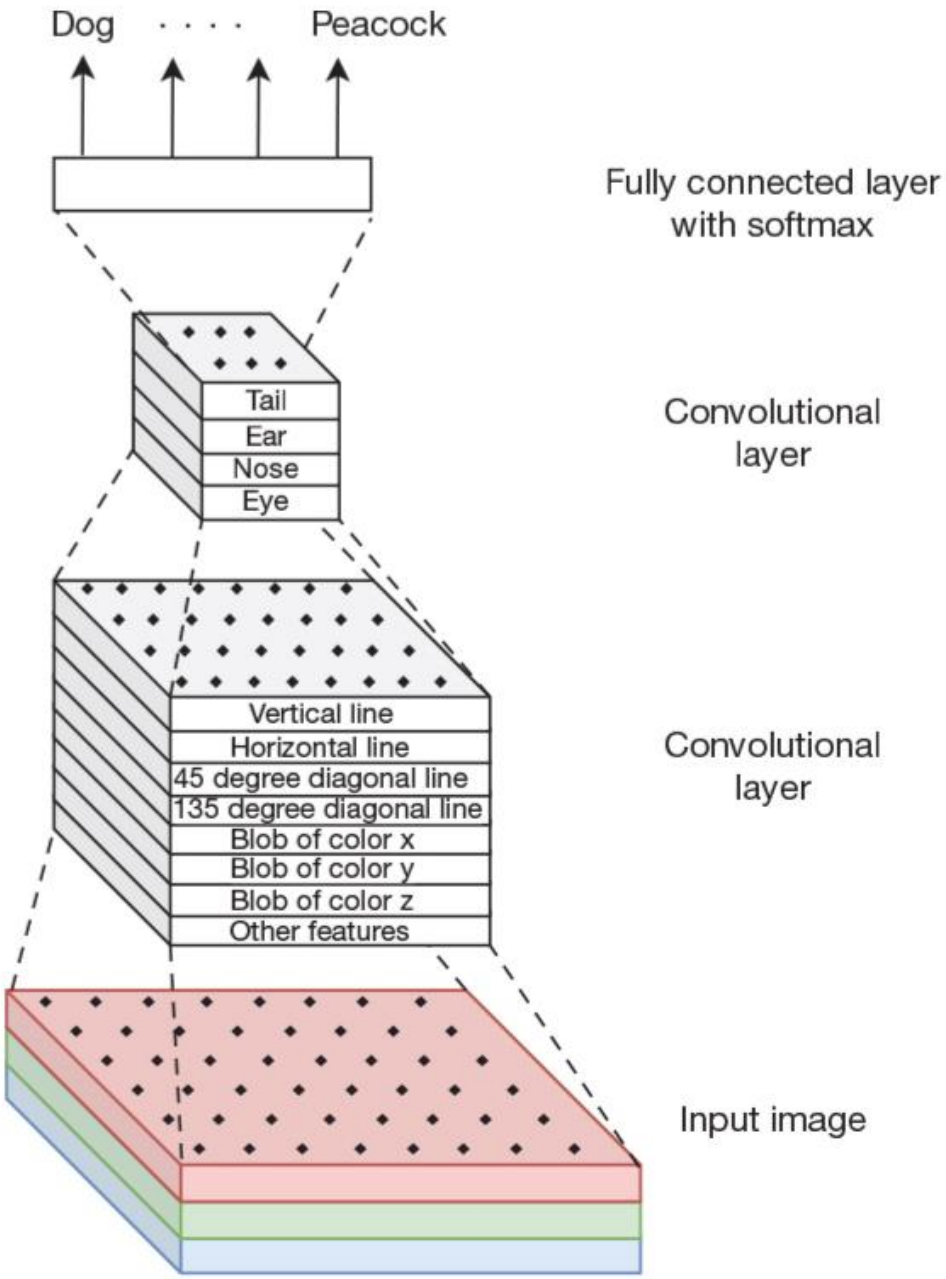
Layer 1 finds edges

Layer 2 finds shapes

Layer 3 finds digit sections

Layer 4 identifies full digits

The image below shows a network identifying dog vs peacock:



While the fully connected NN learns by looking at every pixel at once, the CNN learns by looking at small pieces and building structures.

In the related code, we run multiple Epochs with early stopping. The algorithm continues running Epochs until the model fails to improve. Once it has identified the best Epoch, it uses the previously found best weights. Finally, since we are tuning a hyperparameter (Epochs) we split the train data into a train and validation set. Model output:

Epoch 01 | train_loss=0.2553 | val_loss=0.0777 | val_acc=0.9778

Epoch 02 | train_loss=0.0646 | val_loss=0.0627 | val_acc=0.9808

Epoch 03 | train_loss=0.0450 | val_loss=0.0464 | val_acc=0.9850

Epoch 04 | train_loss=0.0357 | val_loss=0.0435 | val_acc=0.9860

Epoch 05 | train_loss=0.0272 | val_loss=0.0438 | val_acc=0.9870

Epoch 06 | train_loss=0.0225 | val_loss=0.0366 | val_acc=0.9900

Epoch 07 | train_loss=0.0182 | val_loss=0.0362 | val_acc=0.9887

Epoch 08 | train_loss=0.0139 | val_loss=0.0425 | val_acc=0.9880

Epoch 09 | train_loss=0.0124 | val_loss=0.0452 | val_acc=0.9888

Epoch 10 | train_loss=0.0113 | val_loss=0.0505 | val_acc=0.9865

Early stopping triggered. Best epoch was 7 (best val_loss=0.0362). Restoring best weights.

After restore | val_loss=0.0362 | val_acc=0.9887

Test accuracy (CNN): 0.9906

Reinforcement Learning

Reinforcement Learning (RL) is a family of powerful algorithms which are trained to maximize reward within an environment. These models are useful in situations with a lot of inputs and complex outcome structures. For example, many models have been built to play computer games (e.g. racing simulators) at a far higher level than humans can play. By rewarding the agent for making forward progress, the agent learns the best actions to take based on the information presented to it (e.g. an image of the oncoming racetrack environment).

One example insurance use case for RL is optimizing both renewal book pricing accuracy and profitable new business generation. Another use case could be coverage litigation, in which the agent acts as a sequential negotiation assistant (perhaps paired with an LLM).

The example used in the code will be the classic CartPole example, where the agent will balance an upright pole for as long as possible.

Some RL terms to be familiar with:

Agent – the model (aka policy)

Environment – CartPole simulator which has physics built in

State (observation) – the information presented to the agent. For CartPole:

Position

Velocity

Pole Angle

Pole Angular Velocity

Action – choices for what to do. For CartPole: 1 push left or right

Reward – points scored for doing the correct thing (keeping the pole upright for one time step)

Penalty – negative points scored for doing undesirable intermediate actions or intermediate results

Episode – length of the run; either failure or the time limit for CartPole

While supervised learning uses labels/targets, RL uses rewards and consequences. The agent explores and learns via trial and error.